Student: Chun Ho Chui
Student ID: 202030843
Module: COMP3011 - Web Services and Web Data
GitHub repository: https://github.com/errorstd/stock-valuation-api.git
API documentation: https://github.com/errorstd/stock-valuation-api/blob/main/risk-api/docs/API-Documentation.pdf
Presentation slides: https://shorturl.at/NOC4Y (Canva) ;
https://github.com/errorstd/stock-valuation-api/blob/main/risk-api/docs/COMP3011%20Web%20Service%20Presentation.pptx

STOCK VALUATION & RISK ANALYTICS API

Technical Report

1. PROJECT OVERVIEW

1.1 Introduction

This RESTful API provides financial analysis tools for stock market investors, enabling:

- Stock valuation analysis (undervalued/overvalued identification)
- Risk assessment through volatility calculations
- Portfolio performance tracking
- Sector comparison analytics
- Real-time data updates from Yahoo Finance

Technology Stack:

- **Backend:** FastAPI (Python 3.11)
- **Database:** PostgreSQL 16
- **ORM:** SQLAlchemy 2.0
- **Data Source:** Yahoo Finance API (Yfinance)
- **Testing:** pytest with 23 automated tests

1.2 Strategic Pivot: ESG to Financial Valuation

Original Plan: ESG Investment Insights API providing Environmental, Social, and Governance scores.

Critical Issue (March 5, 2026): 100% of ESG data requests returned HTTP 429 errors despite implementing exponential backoff, session management, and rate limiting.

Root Cause: Yahoo Finance deprecated/restricted ESG endpoints. The yesg library (last updated 2023) uses outdated API paths.

Decision: Pivoted to Stock Valuation & Risk Analytics. This maintained technical complexity while using reliable financial data endpoints (100% success rate).

Result: Core competencies (RESTful design, database architecture, data integration) remain unchanged—only domain shifted from ESG to valuation metrics.

2. ARCHITECTURE & DESIGN DECISIONS

2.1 Programming Language: Python 3.11

Justification:

1. **FastAPI Ecosystem -** Automatic OpenAPI documentation (Swagger UI), built-in validation with Pydantic

2. **Financial Libraries -** Mature yfinance wrapper, pandas/numpy for calculations
3. **Developer Productivity -** Rapid prototyping, clear syntax, rich standard library

Alternative Considered: Node.js/Express - **Rejected** due to weaker financial library ecosystem and manual API documentation requirements.

2.2 Web Framework: FastAPI

Justification:

1. **Automatic Documentation -** Swagger UI generated from code, meets coursework requirements automatically
2. **Performance -** 2-3x faster than Flask, async support for concurrent requests
3. **Type Safety -** Pydantic validation catches errors early with clear error messages

Alternative Considered: Flask + Flask-RESTful **- Rejected** due to manual documentation requirements and lack of built-in validation.

2.3 Database: PostgreSQL 16

Justification:

1. **Relational Data Model** - Financial data has clear parent-child relationships (Companies → Financial Metrics, Companies → Stock Prices)
2. **ACID Compliance** - Financial data requires transactional consistency; rollback prevents partial updates
3. **Performance** - Indexing on *symbol, company_id, date* enables efficient joins and aggregations

Alternative Considered: MongoDB - Rejected because relational model better suits financial data structure with foreign key relationships.

2.4 ORM: SQLAlchemy 2.0

Justification:

1. **Type Safety** - Python type hints with IDE autocomplete support
2. **CASCADE DELETE** (Critical Feature) - Proper relationship configuration ensures deleting a company automatically removes all related financial metrics and stock prices
3. **SQL Injection Prevention** - Parameterized queries by default

CASCADE DELETE Configuration:

```
class Company(Base):
    financial_metrics = relationship(
        "FinancialMetric",
        back_populates="company",
        cascade="all, delete-orphan"  # Key fix
    )
```

3. KEY TECHNICAL CHALLENGES & SOLUTIONS

3.1 ESG Data UnavailabilityJustification:

Problem: 100% ESG data requests failed with HTTP 429 errors

Attempted Solutions:

1. Exponential backoff (5s, 10s, 15s delays) **- Failed**
2. Alternative ESG APIs - Required $500/month subscriptions
3. Fallback to industry estimates - Lacked authenticity

Final Solution: Project pivot to Stock Valuation & Risk Analytics using reliable Yahoo Finance financial data.

Lesson Learned: Always validate external API availability BEFORE committing to architecture. Have backup plans for critical data dependencies.

3.2 **CASCADE DELETE & Foreign Key Violations**

Problem: *DELETE /companies* failed with *ForeignKeyViolation: foreign key constraint "financial_metrics_company_id_fkey"*

Root Cause: Related records in *financial_metrics* and *stock_prices* still referenced the deleted *company_id*.

Solution:

1. Configured *cascade="all, delete-orphan*" in SQLAlchemy relationships
2. Changed delete endpoint from raw SQL to ORM delete loop

Before (Broken):

```
db.query(models.Company).delete()  # Raw SQL bypasses cascade
```

After (Fixed):

```
companies = db.query(models.Company).all()
for company in companies:
    db.delete(company)  # ORM delete respects cascade
```

Result: DELETE operations now automatically remove related records. Tested with 150 companies + 10,000 price records - 100% success

4. TESTING & VALIDATION

4.1 Three-Tier Testing Approach

Tier 1: Database Tests *(test_db_connection.py)*

- PostgreSQL connection verification
- CASCADE DELETE functionality
- Foreign key constraint enforcement

Tier 2: API Endpoint Tests *(test_api.py)*

- FastAPI TestClient for HTTP simulation
- Response status code validation (200, 404, 400)
- JSON structure verification

Tier 3: Analytics Logic Tests *(test_analytics.py)*

- Volatility calculation accuracy
- P/E ratio classification
- Risk level determination
- NumPy type conversion

4.2 Test Results

Automated Test Suite: 23 tests, 100% pass rate

- Database test: 4/4 passed
- API tests: 12/12 passed
- Analytics tests: 7/7 passed

Key Test Case: Undervalued Stock Detection

- Endpoint: *GET /analytics/valuation/undervalued?limit=5&max_pe=15*
- Expected: 5 stocks with P/E ≤ 15, sorted ascending
- Result: PASSED (INTC P/E 8.45, manually verified against Yahoo Finance: 8.49)

Performance Testing (Chrome DevTools):

- GET /companies?symbol=AAPL: 45ms avg
- GET /analytics/valuation/undervalued: 82ms avg
- GET /analytics/risk/volatility/AAPL: 67ms avg
- All endpoints < 500ms (excellent for interactive use)

5. LIMITATIONS & FUTURE IMPROVEMENTS

5.1 Current Limitations

1. **Data Source Dependency -** Relies solely on Yahoo Finance without paid backup (risk if access restricted)
2. **No Authentication -** API publicly accessible without rate limiting or user-specific portfolios
3. **Limited Financial Metrics -** Only basic valuation (P/E, EPS); missing DCF analysis, beta, Sharpe ratio
4. **No Real-Time Streaming -** Data static after import; requires manual refresh
5. **Single Currency -** All values assumed USD; no currency conversion

5.2 Future Enhancements

1. **Authentication & Authorization** - OAuth2/JWT tokens, user-specific portfolios, API rate limiting per user
2. **Advanced Analytics** - Discounted Cash Flow (DCF) valuation, beta calculation, Sharpe ratio, technical indicators (RSI, MACD)
3. **Real-Time Features** - WebSocket support for live price updates, push notifications for price alerts
4. **Multi-Source Aggregation** - Alpha Vantage, IEX Cloud as backup providers (99.9% uptime)
5. **Machine Learning** - Price prediction models, anomaly detection, sentiment analysis from news

APPENDIX

6. GENERATIVE AI USAGE DECLARATION

6.1 Tools Used

Primary Tool: Claude (Anthropic) - Claude 3.5 Sonnet

Usage Period:

- **March 2-9, 2026**
- **~50 conversation exchanges across multiple sessions**

Secondary Tool: GitHub Copilot

- Disabled for critical algorithms (volatility, P/E analysis)
- Used only for boilerplate code (imports, docstrings)

6.2 High-Level Usage Examples

Example 1: Understanding CASCADE DELETE (Technical Concept)

- **Prompt:** "What's the difference between cascade='all' and cascade='all, deleteorphan' in SQLAlchemy?"
- **AI Response:** Explained cascade options, provided code examples, mentioned raw SQL bypasses ORM cascade
- **My Actions:** Reviewed SQLAlchemy docs, implemented cascade in all relationships, created test suite, tested with 150 companies
- **Learning Outcome:** Deep understanding of SQLAlchemy cascade semantics and ORM vs raw SQL behavior

Example 2: Financial Volatility Calculation (Domain Knowledge)

- **Prompt:** "What's the standard formula for calculating stock price volatility for risk assessment?"
- **AI Response**: Explained standard deviation of daily returns, provided formula, clarified risk level interpretation (2% = low, 2-4% = moderate, >4% = high)
- **My Actions:** Reviewed SQLAlchemy docs, implemented cascade in all relationships, created test suite, tested with 150 companies
- **Learning Outcome:** Deep understanding of SQLAlchemy cascade semantics and ORM vs raw SQL behavior

Example 3: Strategic Decision on Project Pivot (Critical Thinking)

- **Prompt:** "I've been getting 429 errors on all ESG data requests for 3 days. Should I keep trying or is this a deprecated API? Deadline is in 1 week."
- **AI Response**: Explained Yahoo Finance ESG endpoints are unreliable since 2023, suggested pivot to financial metrics
- **My Actions:** Tested ESG endpoints with 20 symbols (100% failure), validated financial endpoints (100% success), made strategic decision to pivot, refactored codebase (2 days work)
- **Learning Outcome:** Risk management in software development, strategic decisionmaking under time constraints

Example 4: Feature Design - Browse/Search System (Creative Exploration)

- **Prompt:** "What are some useful browse/search features for a stock analysis API beyond basic CRUD?"
- **AI Response**: Suggested category browsing, full-text search, new stocks feed, sector aggregation
- **My Actions:** Designed RESTful endpoints, implemented SQL LIKE queries, created specialized endpoints *(/browse/tech-stocks, /browse/green-energy)*
- **Learning Outcome:** Feature discovery, RESTful API design patterns, user experience considerations

6.3 Analysis on AI Usage Self-Assessment

Self-Assessed Band: 80-89 (High First Class - Outstanding)

Justification: My usage aligns with the 80-89 band description: "High level use of GenAI to aid creative thinking and solution exploration"

Evidence:

1. Creative Thinking - Used AI to brainstorm browse/search features, explored CASCADE DELETE alternatives

2. Thorough Verification - Created 27-test suite, manually validated all calculations against Yahoo Finance

3. Balanced Use - ~50 conversations over 7 days for learning concepts, NOT for writing production code

4. Understanding - Never copy-pasted without comprehension; adapted all suggestions to fit architecture

Why Not 70-79 (Very Good)? Went beyond understanding technology to creative problemsolving and strategic decision-making.

Why Not 90-100 (Exceptional)? Did not explore multiple alternative architectures or use AI for research-level exploration

More logs:
https://www.perplexity.ai/search/so-i-am-planning-to-complete-a-jej1qDGGRhueobPsgoNWkw